

Level-1 Problem 1: Safe Division Calculator

Scenario:

A calculator application must perform division operations safely. If the user enters an invalid value or tries to divide by zero, the system should handle the error gracefully instead of crashing.

Requirements:

1. Create a class **Calculator**.
2. Create a method **Divide(int numerator, int denominator)**.
3. Use **try-catch-finally** blocks to handle errors.
4. If the denominator is zero, display an appropriate error message.
5. Ensure the program continues execution after handling the error.

Technical Constraints:

1. Use **try-catch-finally** blocks for exception handling.
2. Handle **DivideByZeroException**.
3. Ensure the **finally block always executes**.

Expectations:

- Demonstrate proper use of try-catch-finally blocks.
- Display meaningful error messages.
- Ensure program stability even when errors occur.

Learning Outcome:

- Understand basic exception handling.
- Use try-catch-finally blocks effectively.
- Handle runtime errors safely.

Sample Input:

Numerator = 20

Denominator = 0

Sample Output:

Error: Cannot divide by zero
Operation completed safely

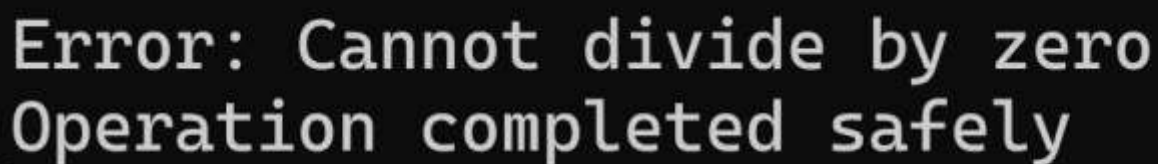
CODE:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace HandsOnDAY20
{
    class Calculator
    {
        public void Divide(int numerator, int denominator)
        {
            try
            {
                int result = numerator / denominator;
                Console.WriteLine("Result: " + result);
            }
            catch (DivideByZeroException)
            {
                Console.WriteLine("Error: Cannot divide by zero");
            }
            finally
            {
                Console.WriteLine("Operation completed safely");
            }
        }
    }

    internal class Program1
    {
        static void Main(string[] args)
        {
            Calculator calc = new Calculator();
            calc.Divide(20, 0);
            Console.ReadLine();
        }
    }
}
```

OUTPUT:



```
Error: Cannot divide by zero
Operation completed safely
|
```

Level-2 Problem 1: Bank Withdrawal System with Custom Exception

Scenario:

A bank wants to implement a withdrawal system that throws a custom exception when a customer attempts to withdraw more money than available in their account.

Requirements:

1. Create a class **BankAccount** with a private field **balance**.
2. Create a method **Withdraw(double amount)**.
3. Create a **custom exception class InsufficientBalanceException**.
4. Throw the custom exception if the withdrawal amount exceeds the balance.
5. Handle the exception in the main program using **try-catch**.
6. Ensure proper cleanup using a **finally block**.

Technical Constraints:

- Use **custom exception classes** by inheriting from Exception.
- Implement **error propagation** by throwing exceptions from methods.
- Handle exceptions in the calling method.

Expectations:

- Demonstrate creation and usage of custom exceptions.
- Implement proper exception handling in calling code.
- Display user-friendly error messages.

Learning Outcome:

- Learn how to create **custom exceptions**.
- Understand **throw and error propagation**.
- Apply structured exception handling in real-world scenarios.

Sample Input:

Balance = 3000
Withdraw = 5000

Sample Output:

Error: Withdrawal amount exceeds available balance

CODE:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace HandsOnDAY20
{
    class InsufficientBalanceException : Exception
    {
        public InsufficientBalanceException(string message) : base(message)
        {
        }
    }

    class BankAccount
    {
        private double balance;

        public BankAccount(double balance)
        {
            this.balance = balance;
        }

        public void Withdraw(double amount)
        {
            if (amount > balance)
            {
                throw new InsufficientBalanceException("Error: Withdrawal amount exceeds available balance");
            }

            balance -= amount;
            Console.WriteLine("Withdrawal successful. Remaining Balance: " + balance);
        }
    }

    internal class Program2
    {
        static void Main(string[] args)
        {
            BankAccount account = new BankAccount(3000);

            try
            {
                account.Withdraw(5000);
            }
            catch (InsufficientBalanceException ex)
            {
                Console.WriteLine(ex.Message);
            }
            finally
            {
                Console.WriteLine("Transaction completed");
            }
        }
    }
}
```

OUTPUT:

```
Error: Withdrawal amount exceeds available balance  
Transaction completed  
|
```